

"...one of the most highly regarded and expertly designed C++ library projects in the world."
— Herb Sutter and Andrei Alexandrescu, [C++ Coding Standards](#)

[q Search...](#)

Queries

Queries return Values which meet some predicates. Currently supported are three types of predicates:

- spatial predicates - spatial conditions that must be met by stored Value and some Geometry,
- distance predicates - distance conditions that must be met by stored Value and some Geometry,
- user-defined unary predicate - function, function object or lambda expression checking user-defined condition.

For example queries may be used to retrieve Values:

- intersecting some area but not within other area,
- are nearest to some point,
- overlapping a box and has user-defined property.

Performing a query

There are various ways to perform a query. They are presented below. All of them return Values intersecting some region defined as a Box.

Member function call

```
std::vector<Value> returned_values;
Box box_region(...);
rt.query(bgi::intersects(box_region), std::back_inserter(returned_values));
```

Free function call

```
std::vector<Value> returned_values;
Box box_region(...);
bgi::query(rt, bgi::intersects(box_region), std::back_inserter(returned_values));
```

Range generated by operator|

```
Box box_region(...);
BOOST_FOREACH(Value & v, rt | bgi::adaptors::queried(bgi::intersects(box_region)))
    ; // do something with v
```

Query iterators returned by member functions

```
std::vector<Value> returned_values;
Box box_region(...);
std::copy(rt.qbegin(bgi::intersects(box_region)), rt.qend(), std::back_inserter(returned_values));
```

Query iterators returned by free functions

```
std::vector<Value> returned_values;
Box box_region(...);
std::copy(bgi::qbegin(rt, bgi::intersects(box_region)), bgi::qend(rt), std::back_inserter(returned_values));
```

Spatial queries

Queries using spatial predicates return Values which are related somehow to some Geometry - box, polygon, etc. Names of spatial predicates correspond to names of Boost.Geometry algorithms (boolean operations). Examples of some basic queries may be found in the tables below. The query region and result Values are orange.

intersects(Box)	covered_by(Box)	disjoint(Box)	overlaps(Box)	within(Box)

intersects(Segment)	intersects(Box)	disjoint(Box)	intersects(Box)	disjoint(Box)

Spatial predicates are generated by functions defined in `boost::geometry::index` namespace.

```
rt.query(index::contains(box), std::back_inserter(result));
rt.query(index::covered_by(box), std::back_inserter(result));
rt.query(index::covers(box), std::back_inserter(result));
rt.query(index::disjoint(box), std::back_inserter(result));
rt.query(index::intersects(box), std::back_inserter(result));
rt.query(index::overlaps(box), std::back_inserter(result));
rt.query(index::within(box), std::back_inserter(result));
```

All spatial predicates may be negated, e.g.:

```
rt.query(!index::intersects(box), std::back_inserter(result));
// the same as
rt.query(index::disjoint(box), std::back_inserter(result));
```

Nearest neighbours queries

Nearest neighbours queries return Values which are closest to some Geometry. The examples of k-NN queries are presented below. 5 Values nearest to the Geometry are orange.

nearest(Point, k)	nearest(Box, k)	nearest(Point, k)	nearest(Box, k)	
nearest(Segment, k)	nearest(Point, k)	nearest(Box, k)	nearest(Segment, k)	nearest(Segment, k)

To perform the knn query one must pass the nearest predicate generated by the `nearest()` function defined in `boost::geometry::index` namespace. For non-point Indexables the shortest distance is calculated using `bg::comparable_distance()` function. The following query returns k Values closest to some Point in space.

```
std::vector<Value> returned_values;
Point pt(/*...*/);
rt.query(bgi::nearest(pt, k), std::back_inserter(returned_values));
```

The same way different query Geometries can be used:

```
Box box(/*...*/);
rt.query(bgi::nearest(box, k), std::back_inserter(returned_values));

Segment seg(/*...*/);
rt.query(bgi::nearest(seg, k), std::back_inserter(returned_values));
```



Note

In case of k-NN queries performed with `query()` function it's not guaranteed that the returned values will be sorted according to the distance. It's different in case of k-NN queries performed with query iterator returned by `qbegin()` function which guarantees the iteration over the closest Values first.

User-defined unary predicate

The user may pass a UnaryPredicate - function, function object or lambda expression taking const reference to Value and returning bool. This object may be passed to the query in order to check if Value should be returned by the query. To do it one may use `index::satisfies()` function like on the example below:

```
bool is_red(Value const& v)
{
    return v.is_red();
}

struct is_red_o
{
    template <typename Value>
    bool operator()(Value const& v)
    {
        return v.is_red();
    }
}

// ...

rt.query(index::intersects(box) && index::satisfies(is_red),
        std::back_inserter(result));

rt.query(index::intersects(box) && index::satisfies(is_red_o()),
        std::back_inserter(result));

rt.query(index::intersects(box) && index::satisfies([](Value const& v) { return v.is_red(); }),
        std::back_inserter(result));
```

`satisfies()` may be negated, e.g.:

```
bool is_red(Value const& v) { return v.is_red(); }
bool is_not_red(Value const& v) { return !v.is_red(); }

// ...

rt.query(index::intersects(box) && index::satisfies(is_red),
        std::back_inserter(result));
// the same as
rt.query(index::intersects(box) && !index::satisfies(is_not_red),
        std::back_inserter(result));
```

Passing set of predicates

It's possible to use some number of predicates in one query by connecting them with operator `&&` e.g. `Pred1 && Pred2 && Pred3 && ...`

These predicates are connected by logical AND. Passing all predicates together not only makes possible to construct advanced queries but is also faster than separate calls because the tree is traversed only once. Traversing is continued and Values are returned only if all predicates are met. Predicates are checked left-to-right so placing most restrictive predicates first should accelerate the search.

```
rt.query(index::intersects(box1) && !index::within(box2),
        std::back_inserter(result));

rt.query(index::intersects(box1) && !index::within(box2) && index::overlaps(box3),
        std::back_inserter(result));
```

It's possible to connect different types of predicates together.

```
index::query(rt, index::nearest(pt, k) && index::within(b), std::back_inserter(returned_values));

BOOST_FOREACH(Value & v, rt | index::adaptors::queried(index::nearest(pt, k) && index::covered_by(b)))
    ; // do something with v
```

Iterative queries

The query performed using query iterators may be paused and resumed if needed, e.g. when the query takes too long, or may be stopped at some point, when all interesting values were gathered. The query iterator is returned by `qbegin()` member function which requires passing predicates, like `query()` member function.

```
for ( Rtree::const_query_iterator it = tree.qbegin(bgi::nearest(pt, 10000)) ;
      it != tree.qend() ; ++it )
{
    // do something with value
    if ( has_enough_nearest_values() )
        break;
}
```



Warning

[Warning] The modification of the `rtree`, e.g. insertion or removal of Values may invalidate the iterators.

Inserting query results into another R-tree

There are several ways of inserting Values returned by a query into another R-tree container. The most basic way is creating a temporary container for Values and insert them later.

```
namespace bgi = boost::geometry::index;
typedef std::pair<Box, int> Value;
typedef bgi::rtree< Value, bgi::linear<32, 8> > RTree;

RTree rt1;
/* some inserting into the tree */

std::vector<Value> result;
```

```
rt1.query(bgi::intersects(Box(*...*/)), std::back_inserter(result));  
RTree rt2(result.begin(), result.end());
```

However there are other ways. One of these methods is mentioned in the "Creation and modification" section. The insert iterator may be passed directly into the query.

```
RTree rt3;  
rt1.query(bgi::intersects(Box(*...*/)), bgi::inserter(rt3));
```

You may also pass the resulting Range directly into the constructor or `insert()` member function using Boost.Range adaptor syntax.

```
RTree rt4(rt1 | bgi::adaptors::queried(bgi::intersects(Box(*...*/))));
```

Copyright © 2009-2024 Barend Gehrels, Bruno Lalande, Mateusz Loskot, Adam Wulkiewicz, Oracle and/or its affiliates
Distributed under the Boost Software License, Version 1.0. (See accompanying file LICENSE_1_0.txt or copy at http://www.boost.org/LICENSE_1_0.txt)